Architecture: Credit Risk Portfolio Monitor

This document shows how the data moves from the raw CSV files to the dashboard, where both AI features sit, which library I used at each stage, where the LLM is called and exactly what data is sent to it. **Page 1 covers everything the brief asks for**; page 2 is an appendix with the deeper detail. Everything is written in Python 3.12 (managed with uv), and one codebase serves both the Streamlit app and the REST API.

1. End-to-end flow

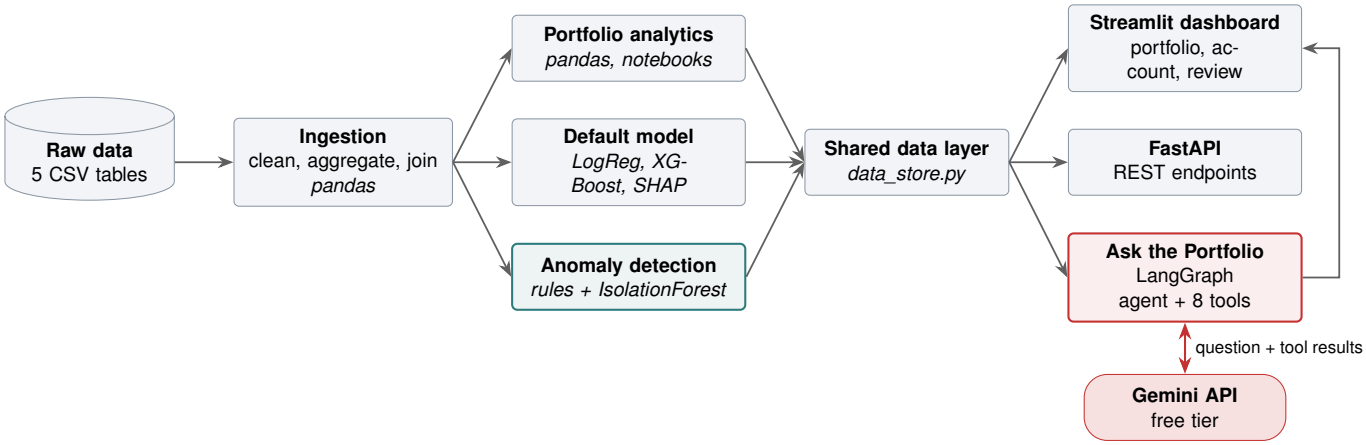


Figure 1: Raw data → ingestion → analytics → model → dashboard, with both AI features

Stage	What happens	Tools	Code / output
Raw data	5 tables: customers (10,000), loan accounts (14,685, holds the target), bureau lines (22,979), monthly repayments (176,220), collections events (8,635).	CSV	`data/raw/`
Ingestion	Key and referential checks; cleaning (M/F labels, group-median imputation with missing flags); each child table aggregated to its key, then LEFT JOIN onto loans. Row count asserted, no rows dropped.	pandas, pyarrow	`src/ingestion.py` → `loan_level.parquet`
Analytics	Default rate, exposure and risk lift by income, employment, region, age and loan type; cross-tabs.	pandas, matplotlib, Jupyter	`src/portfolio.py`, notebooks 01–03
Default model	Logistic regression (champion) and XGBoost (challenger); split by customer; 5-fold grouped CV; AUC / KS / Gini; SHAP drivers; calibrated PD and four risk bands.	scikit-learn, xgboost, shap	`src/model.py` → `models/`
Dashboard and API	Portfolio, account, anomaly review, model and data-quality views with filters and drill-down; chat panel on the right; REST endpoints for the same logic.	Streamlit, Plotly, FastAPI	`app/app.py`, `src/api/`

2. The two AI features

AI feature 1 – Ask the Portfolio (agentic LLM)

A **LangGraph ReAct agent** (`create_react_agent`) runs on **Gemini 3.5 Flash-Lite (free tier)** with **8 tools**. Gemini only decides which tool to call and writes the final sentence. The tool runs locally and computes every figure with pandas or the saved model, and a grounding check confirms that each number in the answer exists in the tool output.

Where the LLM is called: inside the agent, normally twice per question – once to choose the tool and once to write the answer from the tool result. **What is sent to Gemini:** the analyst's question, the last few chat turns, the system prompt, the tool descriptions and the JSON result of the tools it called (aggregated figures, or the one account asked about). **Never sent:** raw customer tables or row-level data for the book. The key lives in `.env` or Streamlit secrets.

AI feature 2 – Anomaly detection (no LLM)

`src/anomaly.py` (pandas, scikit-learn) flags accounts whose behaviour is unusual against their **own history** (last 3 months vs earlier) or their **peer segment** (same loan type × income band). An IsolationForest ranks the flags. Each flag has plain-English reasons, and the analyst confirms or dismisses it in the dashboard; the decision is logged.

Appendix: detailed view

3. Ask the Portfolio – inside the agent

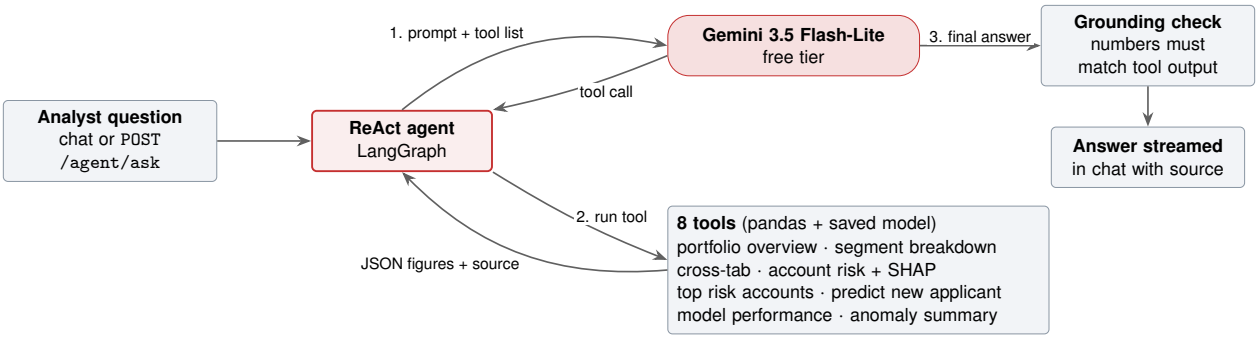


Figure 2: LangGraph ReAct loop: Gemini chooses, tools compute, Python verifies

How one question flows

1. The analyst types a question; it gets a trace id and the last 6 chat turns are added as memory.
2. **LLM call 1:** Gemini reads the system prompt and tool schemas and returns a tool call, e.g. `get_segment_breakdown(loan_type)`.
3. The tool runs locally in about 10 ms and returns JSON with the figures and a `source`.
4. **LLM call 2:** Gemini writes 2–4 sentences from that JSON only, or calls another tool.
5. Python checks every number against the tool output; unverified figures are flagged.
6. The answer streams into the chat (about 1–4 s end to end). Out-of-scope questions get a `DECLINED:` reply.

Tool catalogue

Tool	Returns
`get_portfolio_overview`	Accounts, default rate, exposure
`get_segment_breakdown`	Rate, exposure, lift by a dimension
`get_cross_tab`	Any two dimensions
`get_account_risk`	PD, band, top SHAP drivers
`get_top_risk_accounts`	Highest-PD accounts
`predict_default_probability`	Score for a new applicant
`get_model_performance`	AUC / Gini / KS, top features
`get_anomaly_summary`	Flag counts and reasons

4. Anomaly detection logic

- **Own history:** average DPD in the last 3 months is 20+ days above the earlier average, or 2+ missed EMIs in the last 3 months after a clean record.
- **Peer segment:** robust z-score (median / MAD) of 4 or more within loan type × income band, on repayment DPD, missed-payment rate, bureau DPD, bureau utilisation and write-offs.
- **Ranking and result:** IsolationForest over the same features orders the list. 938 accounts (6.4%) are flagged and they default at 25.2% against 16.1% for the book.

5. Deployment and serving

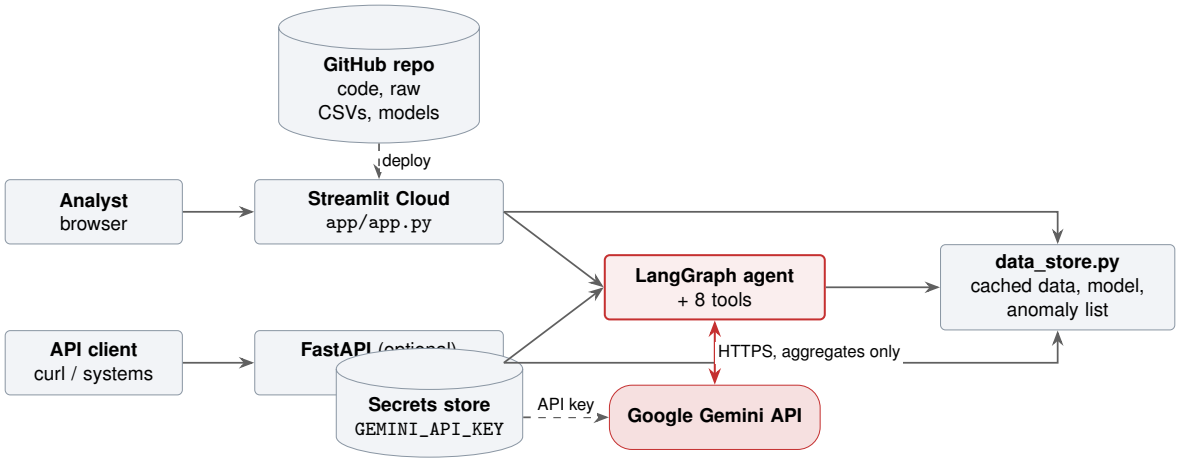


Figure 3: Hosted on Streamlit Community Cloud; FastAPI exposes the same logic

- Streamlit Cloud runs `app/app.py` and calls the agent and model in-process, so the hosted link needs no separate API server.

- The trained models are committed in `models/`; on first start the app rebuilds `loan_level.parquet` from `data/raw/` in a few seconds. `data_store.py` caches the data, model and anomaly list for every surface.

6. Reliability and security

- **Logging:** every request is logged to the terminal with a trace id across UI, API, agent and tools, including the duration of each LLM call and tool run.
- **Errors:** LLM failures are classified (`llm_rate_limited`, `llm_overloaded`, `llm_timeout`, `llm_auth_failed` and so on) and shown to the analyst in plain words; retries are capped and calls time out after 30 s.
- **Data and secrets:** only aggregates or one requested account reach the LLM; the API key is never in code or in the repo. Without a key the dashboard keeps working and only the chat is switched off.